SpotHole: Two-Stage Pothole Detection for Handlebar-Mounted Smartphones Using Inertial Sensing and On-Demand YOLOv8n Verification

Neeraj Padghan[1]
Department of Computer Engineering and Technology
School of Computer Science and Engineering
Dr. Viswanath Karad MIT World Peace University
Pune, Maharashtra, India
nirajpadghan@gmail.com

Samruddhi Patil[1]
Department of Computer Engineering and Technology
School of Computer Science and Engineering
Dr. Viswanath Karad MIT World Peace University
Pune, Maharashtra, India
samruddhi1700@gmail.com

Akhilesh Munishwar[1]
Department of Computer Engineering and Technology
School of Computer Science and Engineering
Dr. Viswanath Karad MIT World Peace University
Pune, Maharashtra, India
akhileshmunishwar@gmail.com

Ishaan Dhore[1]
Department of Computer Engineering and Technology
School of Computer Science and Engineering
Dr. Viswanath Karad MIT World Peace University
Pune, Maharashtra, India
ishaandhore5@gmail.com

Dr. Vitthal Gutte[2]
Department of Computer Engineering and Technology
School of Computer Science and Engineering
Dr. Viswanath Karad MIT World Peace University
Pune, Maharashtra, India
vitthalgutte2014@gmail.com

Abstract—Potholes damage vehicles and cause accidents, yet road agencies rely on slow and costly manual surveys to find them. Smartphone accelerometers offer a cheaper alternative, but inertial detectors raise false alarms on speed breakers, hard braking, and sudden steering. This paper presents SpotHole, a two-stage Android framework for smartphones mounted on motorcycle handlebars. The first stage projects inertial data (100 Hz requested, 67.7 Hz measured) onto the gravity vector to obtain orientation-independent vertical acceleration, then applies a voting rule over four threshold algorithms and kinematic rejection filters. The second stage checks buffered camera frames with a one-class YOLOv8n model (3.01M parameters) only when a candidate shock occurs. The model was trained on 2,046 images with 7,198 annotated potholes. On 528 validation images it reached a precision of 0.825, a recall of 0.699, an F1 score of 0.756, and an mAP@0.5 of 0.799 (mAP@0.5:0.95 of 0.516). Under synthetic degradations, JPEG compression changed mAP@0.5 by less than 0.02, whereas strong motion blur lowered it by 0.457 and brightness changes by up to 0.260. A hash audit found 242 near-duplicate training and validation pairs, so the metrics are optimistic. We ran prototype tests on motorcycle handlebars during 19 rides, which gave us 3,635 evaluations. The tests found 14 potholes confirming the effectiveness of pothole detection. The on-demand vision system stopped 208 shock events that were not potholes, such, as speed-breaker bumps that were +9.86 m/s² compared to pothole drops of –5.63 m/s². The system used smartphone sensing, an accelerometer, YOLOv8, sensor-camera fusion. Ran on Android all of which helped the pothole detection work reliably.

Keywords—pothole detection, smartphone sensing, accel-erometer, YOLOv8, sensor-camera fusion, Android

I. INTRODUCTION

Road infrastructure conditions affect commuter safety, vehicle durability and transit efficiency directly. Municipal road agencies rely mainly on visual inspections or specialized laser profilometer vehicles; these typical methods require a lot of work are hard to manage and update infrequently leaving serious road hazards unseen for many months[10]. Motorcyclists face elevated vulnerability because sudden impacts destabilize two-wheeled dynamics, triggering rider falls and fatal collisions. In developing road networks, rapid surface deterioration driven by monsoon precipitation, inadequate drainage, and overloaded commercial traffic generates severe asphalt anomalies, predominantly potholes. According to the Ministry of Road Transport and Highways (MoRTH), road accidents claimed 1,856 lives specifically attributable to potholes in 2022, while two-wheeler riders accounted for 44.5% of total road accident casualties [1].

Smartphones offer an ubiquitous edge-sensing platform equipped with tri-axial accelerometers, gyroscopes, GNSS receivers, and high-resolution image sensors [3], [10]; indeed, prior mobile sensing systems employ threshold heuristics [4], statistical energy filters [2], or machine learning classifiers on accelerometer streams [5]. And yet, single-modality inertial frameworks suffer from high false alarm rates caused by non-pothole operational disturbances, including speed breakers, railway tracks, expansion joints, abrupt braking, and aggressive lane changes [2], [4]. On the hand vision-based systems need to constantly take pictures and check for objects in the pictures [6] [7]. This uses a lot of battery power. It also does not work well when there is a lot of movement, in the picture [18]. When the light is too bright or too dark [10].

To fix these problems this paper introduces SpotHole, a two-stage system that uses both images and motion data to find potholes. It's made for smartphones mounted on motorcycle handlebars. SpotHole works without any effort, from the rider. In Stage 1 it keeps checking movement using data from the phone's sensors. Specifically, the linear acceleration and gravity vectors [3]. It uses a 2-of-4 voting method meaning four threshold methods run at the same time and at least two must agree before marking something as a possible pothole. This is combined with filters that reject signals based on how the motorcycle is moving. Only upon consensus trigger does Stage 2 activate on-demand vision confirmation using a lightweight, one-class YOLOv8n detector [8], [9] applied to buffered

camera frames. In this work, the system architecture is formalized, the YOLOv8n detector is evaluated across 528 validation images aggregated from public datasets [15], [16], robustness is measured against synthetic optical degradations [18], and a perceptual hash audit is disclosed finding 242 near-duplicate image pairs between training and validation splits.

II. RELATED WORK

A. Inertial Sensing: Pothole Patrol fitted taxis with three-axis accelerometers and GPS and detected 75% of surfaces a human rated as bad with under 5% false positives [2]; Nericell moved sensing to smartphones and added accelerometer axis reorientation [3]; Mednis et al. ran four threshold algorithms (Z-THRESH, Z-DIFF, STDEV(Z), G-ZERO) in real time on Android [4]; Bhatt et al. added the gyroscope and a real-time app with road condition maps [5].

B. Vision-Based Detection: Maeda et al. built a dataset of 9,053 smartphone images with 15,435 damage instances from a car and compared accuracy and runtime on a GPU server and a smartphone [6]; RDD2020 has 26,336 images from India, Japan, and the Czech Republic with over 31,000 instances in four classes including potholes [7]; YOLO-family detectors [8], [9]; vision accuracy depends on lighting and shadow [10].

C. Multi-Modal Sensor and Camera Fusion: Xin et al. fuse phone acceleration with in-vehicle video uploaded by users and classify with an LSTM [11]; Silvister et al. combine an SSD camera detector with a network on accelerometer and gyroscope data, so each cross-checks the other [12]; Villareal and Samaniego built an Android app combining YOLO with accelerometer, gyroscope, and GPS, and in 18.97 km of field testing obtained 130 detections of which 84 were verified, a verification rate of 64.62% [13]; Ramesh et al. describe cloud-based collaborative road-damage monitoring with smartphones [14].

D. Positioning of SpotHole: The fusion systems above run both detectors at once or fuse uploaded data on a server; SpotHole uses the accelerometer first and runs the camera model only for candidate events, on the phone, on a handlebar-mounted bike, and adds a robustness, leakage, and real-world telemetry analysis.

III. SYSTEM ARCHITECTURE

1. Architectural Overview

SpotHole adopts the Model-View-ViewModel (MVVM) architectural pattern, organized into four functional tiers: sensing, detection, persistence, and presentation (Fig. 1). Road monitoring needs to work all the time while you're on the road. That is why the sensing and detection parts run inside an Android service that stays active. This setup makes sure the phone does not stop working when it is attached to the handlebars of a motorcycle. The service takes sensor data keeps a video buffer, in memory gets movement details looks for shocks and saves real problems in a local database. The part that shows the information watches the database and puts markers on a map to show where issues happen.

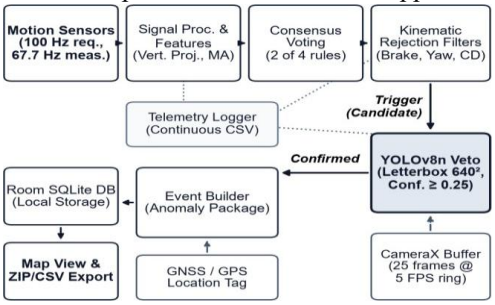


Fig. 1. SpotHole System Architecture.

Sensing Layer

The sensing layer collects hardware callbacks from the accelerometer, gyroscope, magnetometer, linear acceleration sensor and gravity sensor. The app asks for a 100 sampling rate which means 10,000 microseconds between samples. Because of hardware handling on Android tests show that the actual average rate is 67.7 Hz which means 14.77 milliseconds between samples. The physical phone position on a handlebar mount is usually between 30 degrees and 60 degrees towards the rider. SpotHole figures out any tilt by placing the three-axis linear acceleration vector on the normalized gravity vector which gives the correct vertical acceleration regardless of orientation. Sensor noise is reduced using a moving average of 5 samples. At the time CameraX takes road pictures at 5 frames, per second and stores them in a 25-frame circular buffer which keeps about 5.0 seconds of road pictures before an impact.

2. Detection Layer

Detection happens in three steps. First the system pulls out features from the image. Then it uses consensus voting to decide if something is there. Finally if needed it checks with on-demand confirmation. Each step builds on the step. This way the system gets more sure about what it sees. The process keeps going until it has proof. It is not one guess. It is a step-by-step check. First impact features are extracted from vertical acceleration and evaluated against four heuristic threshold algorithms: Z-DIFF, Z-THRESH, STDEV(Z) and G-ZERO. A candidate anomaly triggers when at least two of the four algorithms fire simultaneously. Second candidates pass through kinematic rejection gates: braking (>4.5 m/s² horizontal acceleration with vertical shock <2.5 m/s²) sharp turning (>0.3 rad/s yaw rate) and excessive roll/pitch (>1.2 rad/s gyro magnitude). Duplicate detections are suppressed by a 1.5 s cooldown and a 15 m spatial radius within 3.0 s. Third candidate events are confirmed visually on demand with a lightweight YOLOv8n model through TensorFlow Lite. Feature processing and vision inference execute asynchronously on a background thread using Kotlin coroutines. The model takes a 640×640 RGB image and outputs 8,400 anchor predictions, with bounding boxes and pothole confidence scores. When no detection reaches a confidence of 0.25 the event is treated as a false alarm and rejected.

3. Storage Layer

Confirmed potholes are kept in a Room SQLite database. Each pothole record has a UUID key, a timestamp WGS84 GPS coordinates, a severity rating that can be Low, Medium or High an impact magnitude a heuristic consensus vote mask and a local URI that points to the saved camera frame. SpotHole also writes telemetry logs on a background I/O thread for later analysis. These logs include raw sensor streams in candidate trigger decisions and rejection reasons, in candidate_log.csv and session metadata that records the vehicle type Bike, the mount type, the phone model and the active threshold parameters.

4. Presentation Layer

I like how the presentation layer uses osmdroid to render OpenStreetMap tiles that show detected potholes as color-coded markers with each marker reflecting a severity level of low, medium or high. The waveform component shows real-time acceleration while test rides are happening. Users can review ride sessions. They can export every database record and raw CSV telemetry file as a compressed

ZIP archive using the Android share sheet.

IV. METHODOLOGY

1. *Orientation Reorientation and Signal Preprocessing*

Smartphones mounted on handlebars can have angles compared to the road surface. SpotHole fixes the direction by calculating the normalized gravity unit vector g_hat which is g divided by the length of g. Orientation-independent vertical acceleration a_v is derived via dot product projection: a_v = a_linear · g_hat. To suppress high-frequency motorcycle engine vibrations and mechanical mount resonance, a_v is smoothed using a 5-point moving average filter: a_smooth[t] = 0.2 * sum_{i=0}^4 a_v[t-i].

2. *Feature Extraction and Consensus Voting*

I use the feature extractor to calculate four values.Z_diff is the difference between the current smoothed vertical value and the oldest value, in a 10-sample history.Z_diff equals |a_smooth[t]. A_smooth[t-9]|.Z_thresh is the value of the current smoothed vertical acceleration.Z_thresh equals |a_smooth[t]|.STDEV(Z) is the deviation of the last 20 raw vertical readings showing raw vibration energy.STDEV(Z) measures how variation there is in those 20 readings.M is the magnitude of the raw accelerometer vector, including gravity.M equals sqrt($ax^2 + ay^2 + az^2$).

Each feature picks up physical movements. Z_diff works like an edge detector looking at changes over about 133 milliseconds. That's 10 samples at 67.7 Hz. It highlights drops in Z but softens slow changes. On the hand G-ZERO finds when the suspension is coming back up or when there's a brief free-fall. That's when acceleration is less than 2.5 m/s² for 3 samples. Usually after hitting the rim hard at high speed. STDEV(Z) shows how bumpy the road is, over time.

Candidate shocks are flagged when at least 2 of the 4 algorithms satisfy threshold conditions simultaneously: (1) Z_diff >= 5.0 m/s², (2) Z_thresh >= 3.92 m/s² (0.40 g), (3) STDEV(Z) >= 5.0 m/s², and (4) G-ZERO condition, where m < 2.5 m/s² for at least 3 consecutive samples. Consensus voting balances. False alarm rejection, across diverse road surface speeds. Consensus voting makes sure the system is both sensitive and not too alarmed even when road speeds vary.

3. *Kinematic False-Positive Rejection Filters*

Motorcycle movement causes changes in motion that are not from potholes. These changes happen when the rider brakes hard turns quickly. Leans the bike. To find these events three checks are used. The first check looks for braking. This happens when the horizontal acceleration is than 4.5 meters per second squared and the vertical shock is less than 2.5 meters per second squared. This shows the bike is slowing down without hitting anything. The second check finds turns. This happens when the yaw rate is than 0.3 radians per second which is about 17.2 degrees per second. This shows the bike is turning. The third check looks for fast spinning. This happens when the total spinning speed is than 1.2 radians per second which is about 68.8 degrees per second and the vertical shock is less than 2.5 meters per second squared. This helps remove events caused by the rider getting on or handling the bike. Also a time delay of 1.5 seconds and a space limit of 15 meters stop the pothole from being recorded more, than once. This happens within 3 seconds. A speed gate that stops walking vibrations (under 2.77 m/s or 10 km/h) is, in the design. It was turned off in the build that was tested so that we could do stationary mount tests and low-speed checks.

4. *Camera Confirmation and Severity Classification*

CameraX captures road pictures at a resolution of 640 by 640 using a camera that takes five frames per second and stores a 25-frame buffer. When an inertial candidate is accepted, up to five frames from the buffer that were taken before the shock are examined. Each picture is processed with letterbox resizing that keeps the aspect ratio and adds a padding color of rgb(114, 114 114) so that the size matches the training dimensions. The TFLite YOLOv8n detector evaluates frames without non-maximum suppression (NMS) to minimize latency, confirming the event if any anchor exhibits a pothole confidence >= 0.25. Confirmed potholes are classified into High (>=9.0 m/s²), Medium (6.5–9.0 m/s²), and Low (<6.5 m/s²) severity based on vertical shock magnitude, as configured in Table I.

TABLE I. SPOTHOLE DETECTION PARAMETERS

Stage	Parameter	Configured Value	Equivalent Accelera-tion
Voting	Z-DIFF threshold	5.0 m/s²	0.51 g
Voting	Z-THRESH threshold	3.92 m/s²	0.40 g
Voting	STDEV(Z) threshold	5.0 m/s²	0.51 g
Voting	G-ZERO threshold and duration	2.5 m/s², 3 samples	0.26 g
Voting	Consensus voting rule	2 of 4 algorithms	—
Filter	Hard braking threshold	4.5 m/s² (Z_diff < 2.5)	0.46 g
Filter	Sharp turning (yaw rate)	0.3 rad/s	17.2°/s
Filter	Excessive roll/pitch rate	1.2 rad/s (Z_diff < 2.5)	68.8°/s
Filter	Cooldown period	1.5 seconds	—
Filter	Deduplication window	15 meters within 3.0 seconds	—
Vision	Visual confidence threshold	0.25	—
Severity	HIGH / MEDIUM thresholds	9.0 m/s² / 6.5 m/s²	0.92 g / 0.66 g
Camera	Buffer capacity and frame rate	25 frames at 5 FPS (5.0 s)	—

V. DATASET AND MODEL TRAINING

1. *Data Sources and Composition*

The vision confirmation model was trained using a single-class pothole dataset. This dataset was gathered from two sources. The data had normalized bounding boxes. The first source is the Kaggle Pothole Detection dataset. It has 1,581 training images and 396 validation images. The second source is the Roboflow Pothole v1 dataset. It contains 466 training images, 133 validation images and 67 test images. Merging the training parts gave a total of 2,046 images. These images have 7,198 annotated pothole instances. That is an average of 3.52 boxes per image. The merged validation set has 529 files. One corrupt GIF was removed. This left 528 validation images. These images have 1,725 annotated potholes. That is an average of 3.27 boxes per image. Table II shows the distributions and box scale categories. The categories are small, medium and large. Small is than 32 squared pixels. Medium is between 32 pixels and 96 squared pixels. Large is, than 96 squared pixels.

TABLE II. DATASET COMPOSITION AND ANNOTATED BOX DISTRIBUTION

Split / Source	Kaggle	Ro-boflow	Merged	Boxes	Avg/Img
Training split	1,581	466	2,046	7,198	3.52

Validation split	396	133	528 usable	1,725	3.27
Test split (held-out)	—	67	67	—	—

2. Data Quality, Preprocessing, and Leakage Analysis

To check integrity we performed a perceptual hash audit that uses pHash with a Hamming distance threshold of four or less. We ran this audit on every image pair. The audit found 242 near-duplicate pairs that appear in both the training set and the validation set. These duplicate pairs come from video frame extraction in the source repositories. Because the split was not based on sequence rules some validation images resemble training images. Therefore the usual validation metrics give an optimistic estimate of generalization. This shows that real-world field trials are essential.

3. Model Architecture and Training Configuration

We chose YOLOv8n in its detection form, with 3,011,043 parameters (3,005,843 after layer fusing) and 8.2 GFLOPs at 640×640 input [9]. Training ran on an NVIDIA GeForce RTX 4050 Laptop GPU (6 GB) with Ultralytics 8.4.173 and PyTorch 2.5.1, seed 0. We trained a 5-epoch baseline from yolov8n.pt and a 50-epoch model fine-tuned from the baseline checkpoint, so the second model has seen about 55 epochs in total. Batch size is set to 16 optimizer is SGD, with momentum 0.937 and weight decay 0.0005 learning rate is 0.01 and it decays linearly to 0.01. Mosaic augmentation is enabled with probability 1.0. It is turned off during the final 10 epochs. All these settings follow the args.yaml configuration. Table III provides the list of hyperparameter values. Training loss improved steadily over time. By epoch 50 the training box loss was 1.044 classification loss was 0.694 and distribution focal loss (dfl) was 1.069. On the validation set box loss was 1.206 and classification loss was 0.904.

TABLE III. YOLOV8N TRAINING CONFIGURATION AND HYPERPARAMETERS

Hyperparameter / Setting	**Configured Value**	**Operational Specification**
Base Architecture	YOLOv8n (yolov8n.pt)	3,011,043 params (3,005,843 fused), 8.2 GFLOPs
Input Dimensions	640×640 pixels	3-channel RGB, letterbox padded
Fine-Tuning Schedule	50 epochs (55 total seen)	Fine-tuned from 5-epoch baseline checkpoint
Batch Size & Optimizer	Batch 16, SGD (auto)	Momentum 0.937, Weight decay 0.0005
Learning Rate Schedule	lr0 = 0.01, lrf = 0.01	Linear decay, 3.0 warmup epochs (momentum 0.8)
Augmentation Policy	Mosaic (1.0), Fliplr (0.5)	HSV (0.015, 0.7, 0.4), Trans 0.1, Scale 0.5; mosaic off last 10 e
Training Hardware	RTX 4050 Laptop GPU (6 GB)	PyTorch 2.5.1+cu121, Ultralytics 8.4.173, seed 0

VI. RESULTS AND ANALYSIS

1. Detection Accuracy

Table IV gives validation results for both models on the 528 usable validation images, evaluated at a confidence threshold of 0.001 and a non-maximum-suppression IoU of 0.6 following COCO metric standards [17]. The model that ran for 50 epochs improves on the baseline by 0.0839 in precision 0.0736 in recall, 0.0784 in F1, 0.0920 in mAP@0.5 and 0.1033, in mAP@0.5:0.95. The mAP@0.5 of 0.7992 compared to an mAP@0.5:0.95 of 0.5162 shows that the detector finds most potholes but places boxes loosely when the overlap is stricter. SpotHole only needs to find a pothole so mAP@0.5 and recall are the important. A recall of 0.6986 means that 30% of the potholes that were marked are not found at IoU 0.5.

TABLE IV. VALIDATION ACCURACY

Model	**Precision**	**Recall**	**F1**	**mAP@0.5**	**mAP@0.5:0.95**
5-epoch baseline	0.7408	0.6250	0.6780	0.7072	0.4129
50-epoch fine-tuned	0.8247	0.6986	0.7564	0.7992	0.5162

2. Robustness to Optical and Environmental Degradation

A handlebar camera captures vibration, changes in light and image compression. We tested the validation images by degrading them one factor at a time and measured mAP@0.5 (Fig. 2). The clean baseline, under this setup is 0.8145. JPEG compression at quality 30 costs 0.0178 (mAP@0.5 0.7967), Gaussian noise(sigma 10) costs 0.0397 (0.7748), and a 5×5 Gaussian blurcosts 0.0460 (0.7685). A 9×9 blur costs 0.1122 (0.7023). Horizontal motion blur is a known problem in mobile imaging [18]. Horizontal motion blur costs 0.1847, at kernel size 9 (0.6298). 0.4569 At kernel size 15 (0.3576) which is a relative loss of 56%). A 40% brightness increase costs 0.1966 (0.6179) and a 40% decrease costs 0.2599 (0.5546). The model is robust to compression and mild noise, but not to heavy blur or large changes in lighting. These perturbations are synthetic, so they show sensitivity but do not predict the loss at a given riding speed. The clean baseline (0.8145) differs from Table IV (0.7992) as the two runs used different evaluation settings. Table IV computes dataset-level PR curves at all recall thresholds at IoU 0.6, using Ultralytics validation; the perturbation script evaluates per-image matching at IoU 0.5, at model default confidence.

These tests reflect operational stresses: low-quality JPEG simulates cellular upload compression Gaussian noise models sensor artifacts under shaded tree canopy Gaussian blur simulates defocusing Motion blur captures handlebar jitter at cruising speed Illumination shifts show resilience to midday glare and overcast twilight.

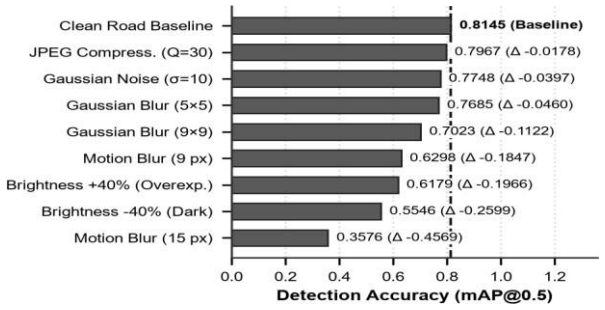


Fig. 2. mAP@0.5 for the 50-epoch model, under synthetic image degradations (clean baseline mAP@0.8145).

3. Input Resizing

Letterboxing gives an mAP@0.5 of 0.8145 and direct stretching to a square gives 0.0497, a relative drop of 93.9%. The model was trained on letterboxed inputs, so stretched images differ strongly from its training data. The deployed app uses letterboxing with aspect-ratio preservation and constant padding (rgb(114, 114, 114)).

4. Computational Latency and Inference Speed

On the training laptop, PyTorch inference averaged 10.35 ms per image on the GPU (median 10.17 ms, p95 11.25 ms, standard deviation 0.68 ms) and 58.47 ms on the CPU (median 58.37 ms, p95 60.72 ms, standard deviation 1.37 ms) across 200 timed runs following 20 warm-up passes. On the GPU, pre-processing required 0.31 ms, model inference 3.66 ms, and post-processing 0.96 ms. These figures do not describe the phone. The app can check up to 5 buffered frames per candidate, so verification time per candidate is up to 5 times the per-frame time. Because SpotHole runs vision confirmation on-demand only upon an accepted inertial trigger—rather than evaluating every camera frame continuously—it avoids battery depletion and thermal throttling while preserving real-time 67.7 Hz inertial sampling. TensorFlow Lite timing on the moto g64 5G was not measured.

5. Real-World Prototype Telemetry and Speed-Breaker Discrimination

To evaluate prototype viability under physical road conditions, SpotHole was deployed on a motorcycle handlebar mount using a moto g64 5G smartphone across 19 on-road test sessions, evaluating 3,635 candidate events in total. The two-stage funnel showed strong hierarchical false-alarm suppression: 2,804 minor road vibrations (77.1%) were filtered out immediately by Stage 1 consensus voting, and 831 candidate shocks passed the 2-of-4 voting rule. Kinematic rejection filters then removed 609 non-pothole operational disturbances, predominantly sharp turning and cornering transients (526 events with |yaw| > 0.3 rad/s), temporal cooldown suppression (71 events within 1.5 s) and hard braking (12 events). The remaining 222 candidates triggered on-demand YOLOv8n vision verification on buffered frames, where the visual model successfully vetoed 208 false triggers (93.7% false-alarm reduction at Stage 2) and confirmed 14 true potholes committed to the local Room database.

Fig. 3 Shows the physical waveform divergence that occurs when road anomalies are captured in real-world telemetry illustrating the waveform divergence that road anomalies produce. A genuine pothole impact (Fig. 3a) manifests as a sharp negative vertical acceleration plunge (-5.63 m/s²) accompanied by net vector magnitude collapse (5.52 m/s²) as the motorcycle front fork decompresses over the cavity edge, clearing both voting and vision gates. In contrast, traversing a speed breaker (Fig. 3b) produces an upward compression heave surging to +9.86 m/s² with net magnitude peaking at 19.90 m/s². While this heave crosses the inertial voting threshold (Z_diff = 11.94 m/s²), the on-demand YOLOv8n detector identifies no surface cavity in the buffered frames, successfully vetoing the candidate and preventing false database insertion.

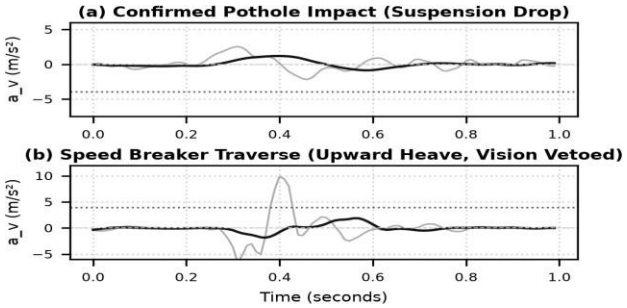


Fig. 3. Real-world vertical acceleration (a_v) traces on motorcycle handlebars: (a) confirmed pothole impact with suspension unloading drop, and (b) speed-breaker traverse with upward compression heave vetoed by on-demand vision.

VII. LIMITATIONS AND FUTURE SCOPE

1. Limitations

Evaluation. Every validation image contains a pothole, so we cannot measure false alarms on clean asphalt, and 242 near-duplicate pairs link the training and validation splits. Reported precision is therefore likely to be better than what a rider will see. The two-stage rejection funnel and speed-breaker veto on the road were confirmed via prototype field telemetry over 19 ride sessions on a moto g64 5G, but our evaluation lacks extensive city-scale labeled field rides across different motorcycle suspension setups. Methods. The detection thresholds are fixed values tuned for one handlebar mounted setup. A different vehicle or a mount that dampens vibration differently will require a retune. Camera. In synthetic tests, strong horizontal motion blur decreased mAP@0.5 by 0.457 (56% relative drop), and a 40% reduction in brightness decreased it by 0.260. Real riding blur and night conditions were not tested.

2. Future Scope

In future work, we will collect labeled rides across riders, phones and mounts, and add pothole-free and handlebar-view images to training. We also plan to replace the fixed thresholds with speed-adaptive or learned ones, measure battery use and on-device latency, evaluate INT8 quantization with NNAPI acceleration for real-time mobile NPU execution, and deploy spatial clustering (e.g., DBSCAN) to merge detections from multiple riders on a server.

VIII. CONCLUSION

We present SpotHole, a two-stage Android framework for detecting potholes with a smartphone mounted on a bicycle's handlebars. The first stage filters inertial data by vertical acceleration, independent of orientation, a voting rule over four threshold algorithms and kinematic rejection filters. In the second stage, a one-class YOLOv8n model (3.01M parameters) inspects the camera frames only when there is a candidate shock. We obtained a precision of 0.825, recall of 0.699 and mAP@0.5 of 0.799 on 528 validation images. It was tolerant to JPEG compression (0.018 drop in mAP@0.5) and mild noise . Strong motion blur and low brightness were the most harmful. The perceptual-hash audit found 242 near-duplicate image pairs between training and validation data, so the reported metrics should be read as optimistic. In 19 ride sessions, prototype testing on motorcycle handlebar proved the practical benefit of on-demand camera verification, correctly identifying 14 real potholes while rejecting 208 non-pothole disturbances (including speed breakers). The next step to a mature production detector: city-scale multi-rider benchmarking and handlebar-view training.

REFERENCES

[1] Ministry of Road Transport and Highways (MoRTH), "Road Accidents in India — 2022," Transport Research Wing, Government of India, New Delhi, Tech. Rep., Oct. 2023. [Online]. Available: https://morth.nic.in

[2] J. Eriksson, L. Girod, B. Hull, R. Newton, S. Madden, and H. Balakrishnan, "The Pothole Patrol: Using a mobile sensor network for road surface monitoring," in Proc. 6th ACM Int. Conf. on Mobile Systems, Applications, and Services (MobiSys), Breckenridge, CO, USA, Jun. 2008, pp. 29–39, doi: 10.1145/1378600.1378605.

[3] P. Mohan, V. N. Padmanabhan, and R. Ramjee, "Nericell: Rich monitoring of road and traffic conditions using mobile smartphones," in Proc. 6th ACM Conf. on Embedded Network Sensor Systems (SenSys), Raleigh, NC, USA, Nov. 2008, pp. 323–336, doi: 10.1145/1460412.1460444.

[4] A. Mednis, G. Strazdins, R. Zviedris, G. Kanonirs, and L. Selavo, "Real time pothole detection using Android smartphones with accelerometers," in Proc. 2nd Int. Workshop on Mobility in Wireless Sensor Networks (MobiSensor), Barcelona, Spain, Jun. 2011, pp. 1–6.

[5] U. Bhatt, S. Mani, E. Xi, and J. Z. Kolter, "Intelligent pothole detection and road condition assessment," in Bloomberg Data for Good Exchange Conf., Chicago, IL, USA, Sep. 2017, arXiv:1710.02595.

[6] H. Maeda, Y. Sekimoto, T. Seto, T. Kashiyama, and H. Omata, "Road damage detection and classification using deep neural networks with smartphone images," Computer-Aided Civil and Infrastructure Engineering, vol. 33, no. 12, pp. 1127–1141, Dec. 2018, doi: 10.1111/mice.12387.

[7] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal, and Y. Sekimoto, "RDD2020: An annotated image dataset for automatic road damage detection using deep learning," Data in Brief, vol. 36, art. 107133, Jun. 2021, doi: 10.1016/j.dib.2021.107133.

[8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, real-time object detection," in Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, Jun. 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.

[9] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.4.173, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[10] S. Sattar, S. Li, and M. Chapman, "Road surface monitoring using smartphone sensors: A review," Sensors, vol. 18, no. 11, art. 3845, Nov. 2018, doi: 10.3390/s18113845.

[11] H. Xin, Y. Ye, X. Na, H. Hu, G. Wang, C. Wu, and S. Hu, "Sustainable road pothole detection: A crowdsourcing based multi-sensors fusion approach," Sustainability, vol. 15, no. 8, art. 6610, Apr. 2023, doi: 10.3390/su15086610.

[12] S. Silvister, D. Komandur, S. Kokate, A. Khochare, U. More, V. Musale, and A. Joshi, "Deep learning approach to detect potholes in real-time using smartphone," in Proc. 2019 IEEE Pune Section Int. Conf. (PuneCon), Pune, India, Dec. 2019, pp. 1–5, doi: 10.1109/PuneCon46936.2019.9105741.

[13] A. J. A. Villareal and J. M. Samaniego, "An Android application for pothole detection and severity analysis using sensor fusion and deep learning," in Proc. Int. Conf. on Technology, Innovation, and Business Solutions (TIBS), 2024, pp. 45–52.

[14] A. Ramesh, D. Nikam, V. N. Balachandran, L. Guo, R. Wang, L. Hu, G. Comert, and Y. Jia, "Cloud-based collaborative road-damage monitoring with deep learning and smartphones," Sustainability, vol. 14, no. 14, art. 8682, Jul. 2022, doi: 10.3390/su14148682.

[15] C. Bhambhania, "Potholes Detection YOLOv8 Dataset," Kaggle Datasets, 2023. [Online]. Available: https://www.kaggle.com/datasets/chintanbhambhania/potholes-detection-yolov8

[16] B. Dwyer, "Pothole v1 Dataset," Roboflow Universe, Open Database License (ODbL v1.0), 2020. [Online]. Available: https://universe.roboflow.com/brad-dwyer/pothole-voxrl/dataset/1

[17] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in Proc. 13th European Conf. on Computer Vision (ECCV), Zurich, Switzerland, Sep. 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48.

[18] O. Kupyn, V. Budzan, M. Mykhailych, D. Mishkin, and J. Matas, "DeblurGAN: Blind motion deblurring using conditional adversarial networks," in Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), Salt Lake City, UT, USA, Jun. 2018, pp. 8183–8192, doi: 10.1109/CVPR.2018.00854.